

NBA Shot Make Probability Model

Hamza Khalilullah

Objective

Estimate the probability that an individual field goal attempt is made. I treated the task as probabilistic binary classification and optimized log-loss, which rewards accurate probabilities and penalizes confident mistakes.

Final result

Model	Features	Holdout log-loss	Observed make rate	Mean predicted
CatBoost	37	0.621746	45.81%	45.60%

The final holdout score was essentially equal to three-fold cross-validation, and the average prediction was within 0.21 percentage points of the observed make rate. This is strong evidence that the pipeline generalized as we wanted it to.

Executive Summary

I built a CatBoost classifier to estimate shot make probability from shot geometry, defensive pressure and motion, play context, and player/team identity. The analysis emphasized probability quality rather than a hard make/miss threshold, so log-loss was the primary selection metric and calibration was treated as a first-class diagnostic.

- Shot distance and shot type establish the baseline difficulty of an attempt. Make rate falls sharply away from the rim, with distinct profiles for dunks, layups, jumpers, and other shot types.
- Defense matters beyond geometry. Closest-defender distance, number of contesters, and defender approach history produced the largest incremental improvement after the baseline shot features.
- Context and identity add smaller but repeatable signal. Shot clock, dribbles, shooter speed, game state, shooter and defender identity, and team/time variables improved cross-validated log-loss.
- The selected 37-feature model achieved 0.621739 out-of-fold log-loss. A focused hyperparameter comparison improved this to 0.621550, and the untouched holdout returned 0.621746.
- The model was well calibrated on the holdout: the mean prediction was 45.60% versus a 45.81% observed make rate, and probability-bin outcomes tracked the 45-degree calibration line.

Analytical approach

I separated an untouched validation set before model-directed analysis. EDA and feature decisions were performed on the training portion; three-fold cross-validation on that portion compared cumulative feature sets and hyperparameters. I used the holdout once at the end for an unbiased check. After selecting the specification, the same features and parameters can be fit on all labeled rows before scoring the competition test set.

Why CatBoost

CatBoost is a strong fit for this NBA dataset since it learns nonlinearities and interactions among continuous features while handling categorical fields (e.g. shot type, shooter, defender, team, season, and game state). Its ordered treatment of categories reduces the need for high-dimensional one-hot encoding. Early stopping also gives a data-driven estimate of the useful number of trees.

Data audit and exploratory analysis

The training data contained 425,719 shot attempts. I checked schema, missingness, value ranges, duplicates, target balance, categorical cardinality, and the internal consistency of spatial and distance variables before modeling. There wasn't much class imbalance, so accuracy-based resampling was unnecessary; preserving the natural make rate is preferred for calibrated probability estimates.

Shot geometry

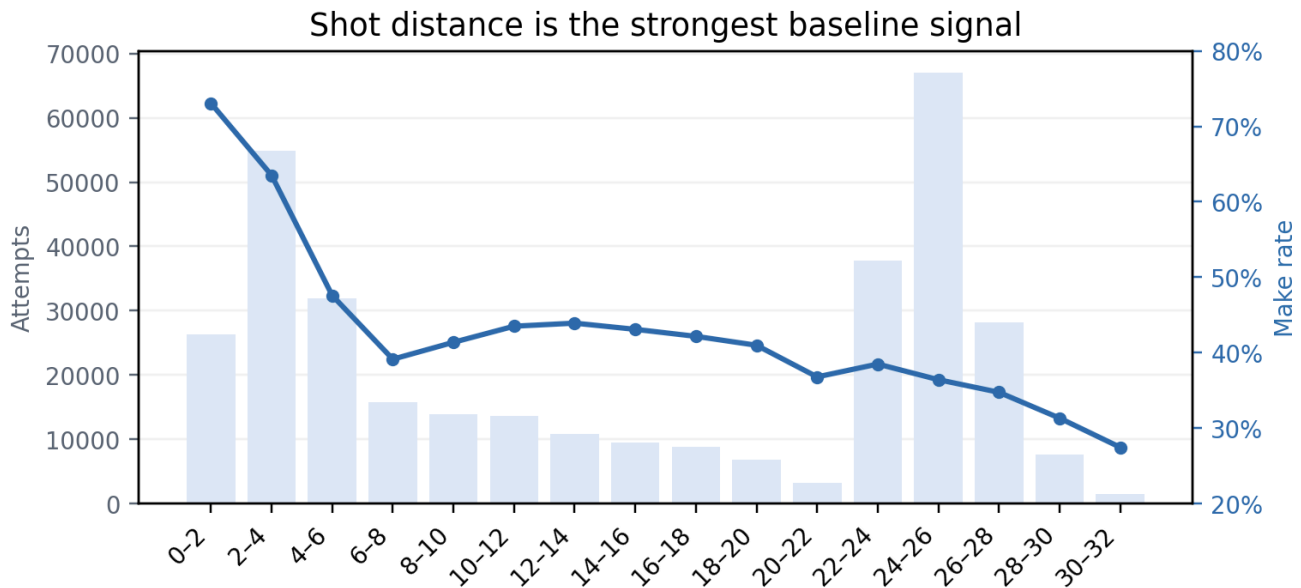


Figure 1 Make rate and attempt volume by two-foot distance band. I omitted the very long-distance bins because they are super sparse and visually unstable

Distance is the clearest first-order signal. Attempts within two feet were made at 73.1%, compared with 36.4% from 24–26 feet and 27.5% from 30–32 feet. The relationship is not completely monotonic since shot type and location composition change with distance. For example, short attempts include both high-value dunks and more difficult post or tip attempts. This motivated retaining both distance and shot type, plus spatial features such as location, absolute lateral position, and rim angle.

Shot type	Attempts	Make rate	Average distance
Jumper	185,257	37.8%	23.0 ft
Layup	92,321	53.5%	3.8 ft
Floater	22,147	41.8%	9.7 ft
Dunk	20,620	88.5%	2.1 ft
Post	11,830	45.8%	8.7 ft
Tip	7,497	48.0%	2.6 ft
Heave	903	10.1%	45.0 ft

Defensive Pressure and Motion

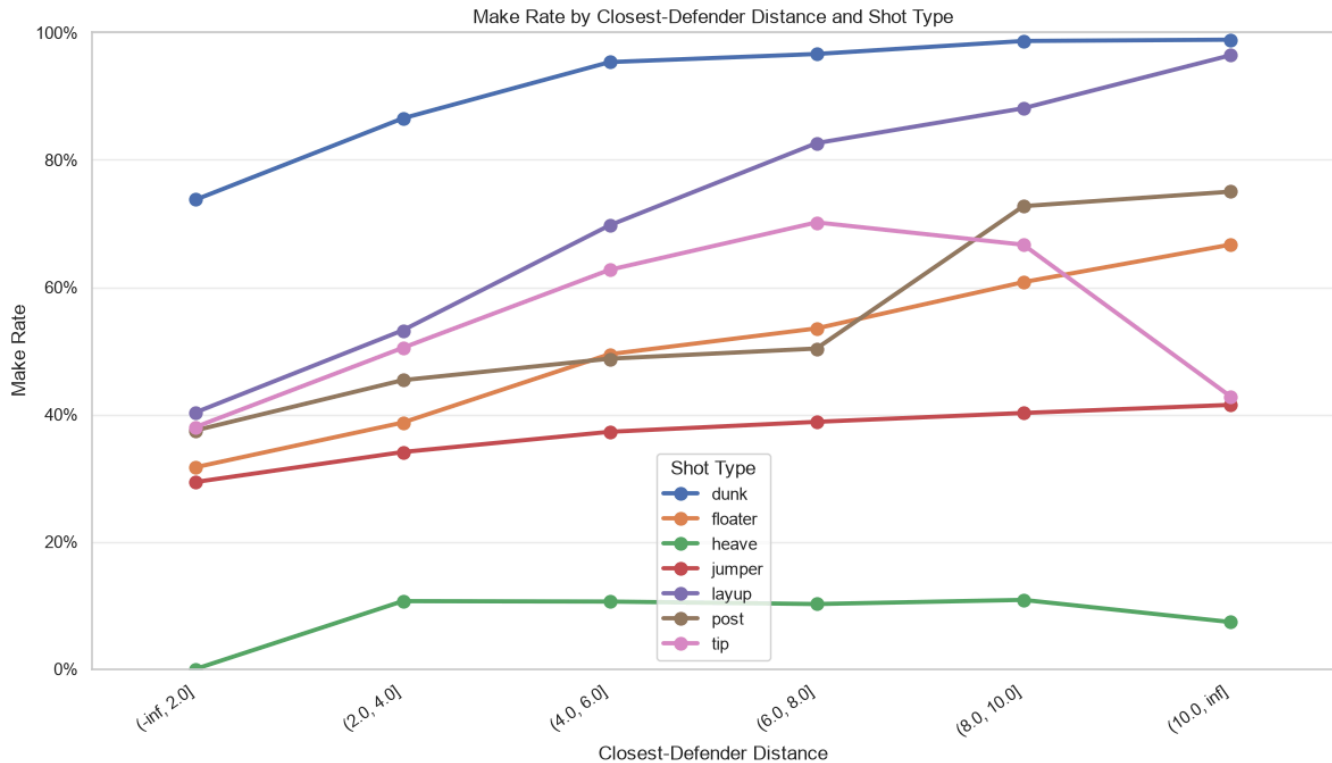


Figure 2 Empirical make rate by closest-defender distance and shot type. Make sure to cautiously interpret sparse extreme bins.

Separation from the closest defender generally increases make probability within shot type, but the level and slope differ a lot across shot types. Specifically, a two-foot change in separation is not equally valuable for a jumper, layup, or dunk. This is the kind of interaction a tree model can learn without needing to manually specify every cross-term.

The data brief distinguished a defender from a contestor. The closest defender is the nearest tracked defender and can exist even when the play is not labeled contested; contesters are defenders meeting the contest definition for the shot. As a result, the final feature set retained both closest-defender variables and contest-specific summaries rather than treating them as interchangeable.

Defender motion

The approach history was stored as string representations of dictionaries, so I parsed and validated the four time-indexed distance values before deriving motion features. Useful summaries included distance at 1.00, 0.75, 0.50, and 0.25 seconds before release, total distance covered, average closing speed, and recent closing amount. Raw categories such as rapidly closing were descriptive, but not necessarily causal: closing speed is confounded with initial distance, shot type, and help rotations. Because of this, I supplied the continuous history to CatBoost and let my model condition on the surrounding context.

Data-quality choices

- The contesters 3 and 4 fields were over 99% missing and contesters 2 was 83.6% missing. I represented these primarily through availability, count, and distance summaries instead of relying on sparse identities.
- Only 78 location records were physically problematic. Because that share is negligible and the values were unlikely to help generalization, they should be removed or set missing using a rule defined before scoring the test data.
- Extreme long-distance attempts were retained when plausible, but conclusions from sparse bins were not used to drive feature decisions.

Feature design

I organized candidate variables into cumulative, basketball-motivated blocks. Each new block was tested on top of the preceding blocks, so its log-loss change measured incremental predictive value instead of a standalone correlation.

Block	Examples	Rationale
A Shot geometry	shot type, distance, x/y location, rim angle	Baseline difficulty and court geometry
B Defensive pressure	closest-defender distance, contested flag, number and distances of contesters	Space and immediate pressure at release
C Defender motion	four approach distances, distance covered, average and recent closing	How pressure developed before release
D Play context	shot clock, dribbles, shooter speed, game state, three-point indicator	Possession state and shot creation
E Shooter identity	shooter identifier	Persistent player skill and shot profile
F Defender identity	closest-defender identifier and related context	Defender-specific effects beyond measured distance
G Team and time	offense/defense team, season, month/time context	System, matchup, and temporal environment

Interpreting observational relationships

Many univariate plots were intentionally treated as descriptive rather than causal. Shooter speed, for example, appeared positively related to make rate overall, but much of that pattern weakened within shot type. Defender closing categories also mixed together distance, rotation behavior, and shot selection. CatBoost can use these variables predictively through interactions, but the resulting feature importance should not be read as the causal effect of changing one variable while holding the play fixed.

Leakage control

Every transformation must be reproducible from information available at or before release and applied identically to train and test. Player or team aggregates, if added later, should be computed out of fold or from prior games only. The current specification avoids target-derived full-sample encodings and uses CatBoost's ordered categorical machinery for identity variables.

Model development and selection

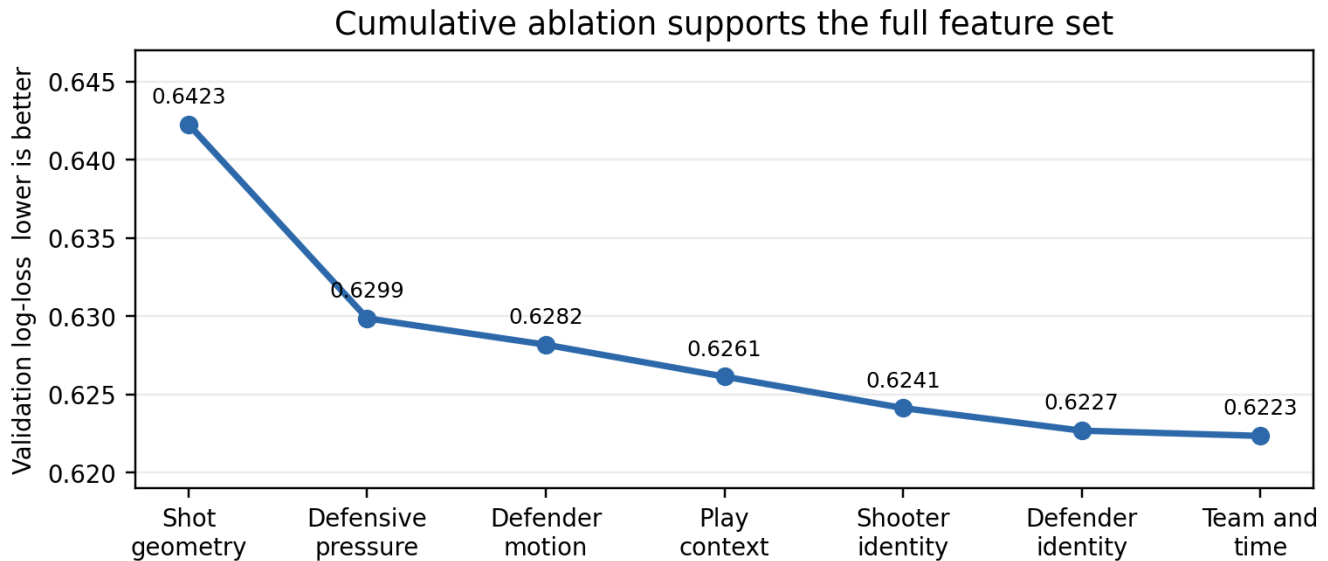


Figure 3 Cumulative feature-group ablation. The defensive-pressure block produced the largest single gain; later blocks added smaller improvements.

The geometry-only model reached 0.642254 log-loss. Adding defensive pressure lowered log-loss by 0.012397, the largest incremental improvement. Motion, play context, and player identity each added useful signal. Team/time variables improved the final screening score by only 0.000333, but the gain appeared in all three finalist folds and was retained.

Cumulative feature set	Features	Log-loss	Gain vs prior
A Shot geometry	7	0.642254	—
B Defensive pressure	15	0.629857	0.012397
C Defender motion	22	0.628175	0.001683
D Play context	30	0.626137	0.002038
E Shooter identity	31	0.624126	0.002011
F Defender identity	33	0.622683	0.001443
G Team and time	37	0.622349	0.000333

The constant-probability baseline log-loss was 0.689626. The full feature set improved this by 0.067276, a 9.76% relative reduction. Because log-loss has no universal “good” threshold, this baseline comparison, cross-validation stability, and holdout replication are more informative than the absolute score alone.

Cross-validation and tuning

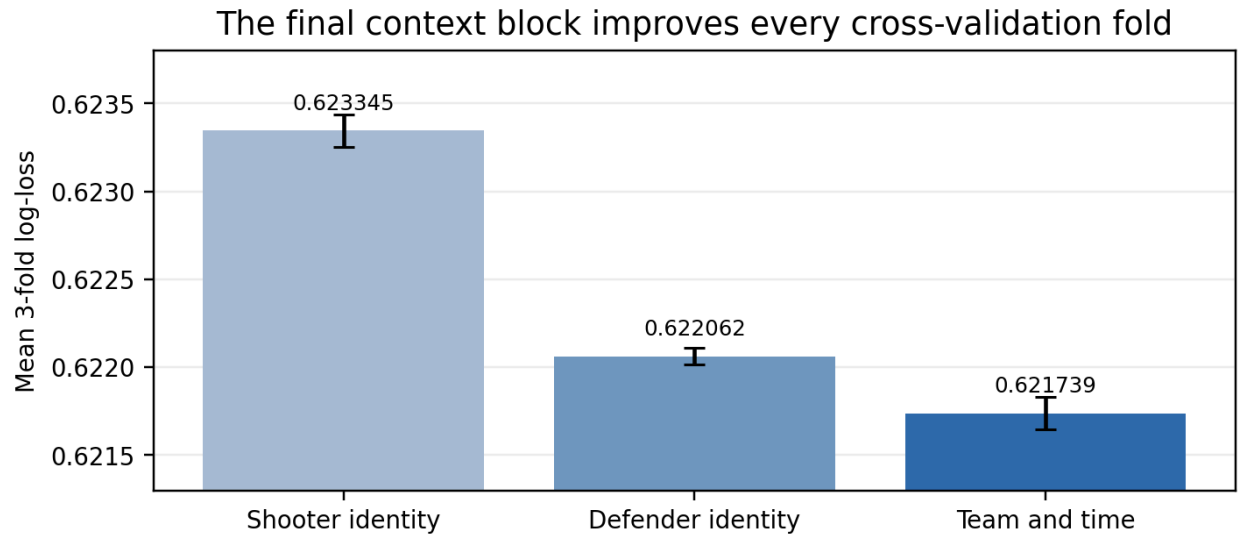


Figure 4 Three-fold cross-validation for the final feature-set candidates. Error bars show the standard deviation across folds; the vertical axis is intentionally zoomed.

Feature set	Mean fold loss	Fold SD	OOF loss	Median trees
Shooter identity	0.623345	0.000093	0.623345	727
Defender identity	0.622062	0.000047	0.622062	1,262
Team and time	0.621739	0.000091	0.621739	1,473

The very small fold-to-fold standard deviations indicate stable estimates under this split. The tree counts are not an additional performance metric; they are the early-stopping iteration at which each fold stopped improving. More trees do not automatically mean a better or more complex model because the required count depends strongly on learning rate. I used the median across folds as a robust training budget for the final fit.

Focused hyperparameter comparison

Candidate	Depth	Learning rate	L2	OOF log-loss	Median trees
Slower learning	7	0.03	8	0.621550	1,870
More regularization	7	0.05	15	0.621613	1,433
Current	7	0.05	8	0.621739	1,473
Depth 6	6	0.05	8	0.621748	1,519
Depth 8	8	0.05	8	0.621762	993
Less regularization	7	0.05	4	0.621800	1,264

The best candidate used depth 7, learning rate 0.03, L2 leaf regularization 8, and 1,870 trees. Its advantage over the initial configuration was modest (0.000189 log-loss), so I would characterize the model as robust to nearby settings rather than claiming a sharply defined optimum. This focused search also limits selection bias relative to an unrestricted sweep.

Final evaluation

Metric	Result	Interpretation
Held-out log-loss	0.621746	Nearly identical to tuned CV result of 0.621550
Observed make rate	45.807%	Actual share of makes in the untouched holdout
Mean predicted probability	45.602%	Only 0.205 percentage points below observed

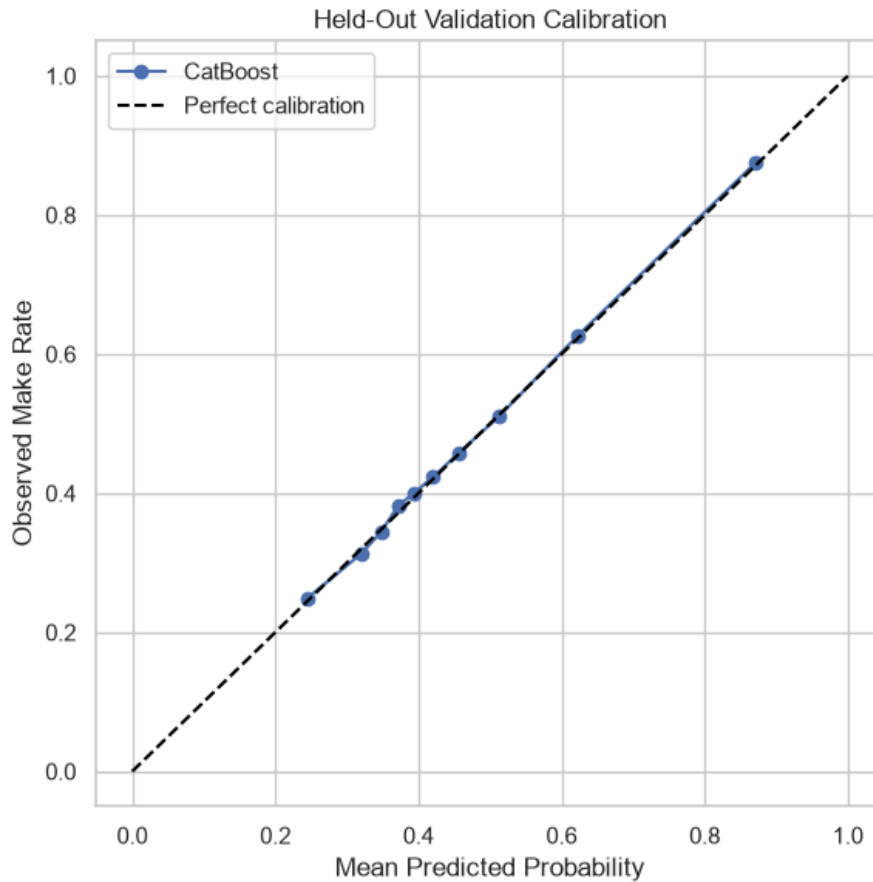


Figure 5 Holdout calibration by probability bin. The curve closely follows perfect calibration across the populated probability range.

The holdout result validates the full workflow. Cross-validation and holdout log-loss differ by only 0.000196, while the aggregate prediction is very close to the observed rate. The calibration curve also shows no obvious systematic overconfidence or underconfidence in the populated bins. A production evaluation should add bin counts and uncertainty intervals, especially near the extremes where relatively few shots occur.

What the output means

The model produces a probability for each attempt, not just a make/miss label. That output can support expected-points calculations, shot-quality summaries, player and lineup evaluation, defensive scheme analysis, or residual analysis comparing actual outcomes with expected makes. For downstream aggregation, calibrated probabilities are more useful than maximizing classification accuracy at a fixed threshold.

Model interpretation

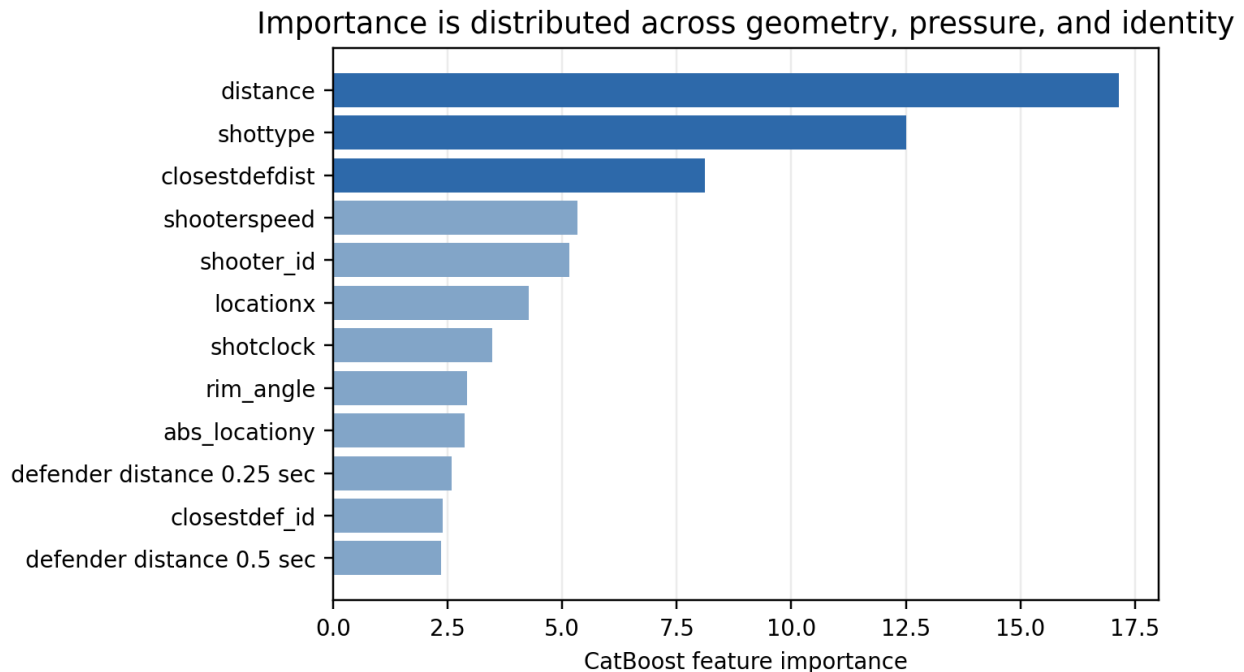


Figure 6 Top CatBoost feature importances from the final model. Importances summarize model usage, not causal effects.

The importance ranking is coherent with my EDA. Distance and shot type lead, followed by closest-defender distance. Shooter speed, shooter identity, spatial location, shot clock, rim angle, and recent defender positions provide complementary context. The mixture of static and dynamic defensive variables suggests that both the amount of space and how that space is changing are useful.

Feature importance is global and can be diluted across correlated variables. For example, closest-defender distance, contest distances, and the approach-history measurements overlap. Their individual ranks should not be interpreted as a complete accounting of defensive value. **With more time, I would add out-of-fold permutation importance and SHAP summaries, plus grouped importance by feature family, to show direction and heterogeneity while respecting correlated inputs.**

Limitations

- Random folds may place shots from the same player, game, or adjacent time period in both training and validation. That is appropriate for interpolation to a similar sample, but a time-based or game-grouped backtest would be a stricter test of future generalization.
- Identity features can capture stable skill and matchup effects, but unseen or low-frequency players and roster changes require careful handling. CatBoost can fall back toward broader patterns, though performance should be reported separately for rare and unseen identities.
- Tracking-derived variables may contain measurement error. The 78 implausible location records and the dictionary-parsing issue illustrate why validation rules must be part of the final pipeline, not one-off notebook edits.
- Global calibration could hide subgroup errors - calibration and log-loss should also be checked by distance, shot type, shooter volume, season, and game state.

Next steps

My model is a good final submission: its feature gains are basketball-plausible, cross-validation is stable, calibration is strong, and the untouched holdout matches the development estimate. With additional time or data, I would prioritize the following:

Priority	Extension	Why it matters
1	Time-based and game-grouped validation	Tests roster drift and prevents same-game dependence from making validation optimistic
2	Out-of-fold player and team history	Adds recent shooting form, role, workload, and matchup context without target leakage
3	Richer tracking geometry	Defender angle, contest hand, help location, shooter orientation, pass origin, and spacing can separate nominal distance from effective pressure
4	Grouped explanation and residual review	SHAP, permutation tests, and video review of large residuals can expose missing basketball context and data errors
5	Calibration stress tests	Reliability by shot type, distance, player volume, and season, with uncertainty intervals and optional post-hoc calibration
6	Small ensemble benchmark	Average CatBoost with a differently structured model only if out-of-fold log-loss and calibration improve consistently

Final training and submission

After freezing the feature logic and hyperparameters, I would refit CatBoost on all labeled data using 1,870 trees, generate make probabilities for the test rows in their original order, and place those probabilities into a copy of the submission template. Before export I would verify row count, identifier/order preservation, absence of missing predictions, probability bounds between 0 and 1, and a plausible prediction distribution relative to training and validation.

Conclusion

Shot geometry explains the largest share of difficulty, defensive pressure supplies the biggest incremental gain, and motion, context, and identity refine the estimate. The agreement among cross-validation, holdout log-loss, aggregate make rate, and calibration show the model is a reliable estimate of shot make probability while leaving clear avenues for harsher time validation and better tracking features.